

Coding Agent

Approach, Assumptions and Limitations

Live app: <https://coding-agent-frontend.onrender.com/>

Code: <https://github.com/Suyash-18/Coding-Agent-Assignment>

1. What was built

A coding agent that a developer can talk to in plain language. Given a request such as "Add input validation to the create-user API and write a test for it", it:

1. understands the request,
2. finds the relevant files in a small sample project (a FastAPI user manager),
3. shows a **plan** and waits for approval,
4. generates the code change and **runs the project's tests** against it,
5. shows the **diff**, the test result and a written **explanation**,
6. applies the change only after a second approval.

It is available three ways, all backed by the same engine: a **Streamlit web app** (deployed), a **FastAPI backend** with a documented HTTP API (deployed), and a **command-line tool**.

2. Approach

Agent flow

The agent is a LangGraph state graph. Each step is a small node with one job, so every stage can be tested on its own and shown to the user as it happens.

Step	Node	What it does
1	input_guard	Rejects over-long, non-coding, prompt-injection and secret-extraction requests before any model call.
2	scan_repo	Lists the repository's files (ignoring <code>.git</code> , <code>.env</code> , binaries and large files).
3	select_files	The model picks the files relevant to the task and says why.
4	read_files	Reads the selected files (size-capped).
5	make_plan	The model writes a plan: which files change and how, plus its assumptions. It can ask for extra files, which are then read (<code>expand_files</code>).
6	plan_approval	Human checkpoint. The run pauses; the user approves, rejects, or adds a note for the model.
7	generate_changes	The model produces the new file contents as structured output.
8	output_guard	Checks the schema, that paths stay inside the repo, file and diff size, secret patterns and protected files.

Step	Node	What it does
9	run_tests	Applies the changes to a temporary copy and runs pytest. On failure, the test output goes back to the model for another attempt (a small, fixed number of retries). Still failing, the result is marked unverified .
10	build_diff	Builds a unified diff of the change.
11	explain	The model explains what changed and why (including a warning if something looks wrong).
12	apply_approval	Human checkpoint. The user applies or declines the change.
13	apply_changes	Writes the files to the real repository.

Key design decisions

- **A workflow with a few model calls, not a free-roaming agent.** The graph is mostly linear, with one retry loop. This keeps behaviour predictable, cheap on a rate-limited free API, and easy to explain and test. The model decides *what* (files, plan, code); plain code decides *how* (file access, diffs, tests, limits).
- **Tools are plain functions, with no LLM inside them.** Listing and reading files, building diffs, applying changes and running tests are deterministic and unit-tested, including deliberate path-escape attacks.
- **Tests are the validation step.** The agent runs the project's own test suite on every proposed change and uses the failures to correct itself. That is the difference between "the model wrote some code" and "the change was checked".
- **Two human approvals.** The user sees the plan before any code is generated, and sees the diff and test result before anything is written. Nothing touches the repository without the second approval, and the CLI's --yes flag never applies a change whose tests failed.
- **One contract, three interfaces.** The engine exposes only `stream_agent()` and `resume_agent()`, which emit a stream of events (`start`, `node_done`, `plan`, `test`, `diff`, `interrupt`, `error`, `done`). The CLI, the API and the web app are thin consumers of that stream, so none of them contains agent logic.
- **Guardrails live inside the graph.** Because the input and output guards are graph nodes, every interface is protected automatically.
- **Everything is logged.** Each run writes a JSON-lines log. The History page replays these logs as a readable summary (task, files, plan, tests, diff, explanation).
- **Models.** The LLM is accessed through Groq. The default is `openai/gpt-oss-120b`; `qwen/qwen3.8-27b` is also selectable. The code never builds a client directly: one factory (`get_llm`) is used everywhere, so models are easy to swap.

Tech stack

LangGraph and LangChain-Groq for the agent; Pydantic for structured model output; Typer and Rich for the CLI; FastAPI (with Server-Sent Events) for the backend; Streamlit for the UI; pytest for the tests; uv for dependencies; Render for hosting.

How it was validated

- Unit tests for the tools layer (including path-escape attempts), each graph node (with a fake LLM), the guardrails (one test per rule plus a list of red-team prompts), the retry loop (a scripted model returns broken code first and fixed code second), and the API.
- End-to-end runs of real tasks through the CLI and the web app, for example "Ensure newly created users have their email stored in lowercase and add a test", and "Add clear docstrings to each route function". Run logs for these are stored in mongo atlas cloud.
- Demo cases that deliberately trip the guardrails to show a clean "blocked" message.

3. Assumptions

- **Target projects are small Python projects with a pytest suite.** Test running is built around pytest. A project with no tests can still be changed, but the result is clearly marked as unverified.
- **The sample project is the main target.** The agent is demonstrated on a small FastAPI user manager included in the repository. Hosted users pick a repository by id from a built-in list instead of supplying a path, because a public server must not read arbitrary folders.
- **Small enough to fit a prompt.** Selected files are read in full (with size caps). The agent has no retrieval or indexing layer, which is a reasonable choice for a handful of files.
- **The user reviews the diff.** The approval steps assume a human reads the plan and the diff. Passing tests are evidence, not proof.
- **A Groq API key is available on the server.** The key is supplied as an environment variable and is never stored in the repository.
- **A single instance serves the app.** Run state is kept in memory, so one backend instance is assumed.
- **Hosting is on Render's free tier** for both services, which shapes several of the limitations below.

4. Limitations

Agent quality

- **Passing tests do not guarantee a good change.** In one run with the Qwen model, the proposed diff dropped several existing, unrelated tests from a test file while the suite still passed. The generated explanation flagged this, and the diff view makes it visible, which is exactly why the second approval exists. Always read the diff before applying.
- **Output depends on the model.** The same task can produce different plans and code on different runs and with different models. The models are small or mid-sized and available on a free tier, so complex multi-file refactors may fail or need a retry.
- **Whole-file generation.** Changes are produced as complete file contents rather than minimal patches, which makes accidental omissions possible and limits how large a file can be.
- **Limited context.** Large files are truncated and very large repositories are not supported, because there is no retrieval layer.

- **Limited retries.** The self-correction loop makes a few attempts only. After that, the change is marked unverified instead of continuing.
- **Python and pytest only.** Other languages and test frameworks are not supported.

Hosted deployment

- **One shared demo repository.** The deployed app edits a single demo/sample_project on the server. A hosted server cannot change a visitor's own machine, so visitors should treat "apply" as a demonstration and use **Download patch** to take the change away. Only one run can write at a time; other visitors see a "repository is busy" message until it finishes.
- **No persistence on the free tier.** Render's free web services lose their local files when they restart, redeploy or spin down. The demo repository and the run history can disappear, and **Scripts, reset_demo** recreates the demo repository.
- **Cold starts.** After 15 minutes without traffic the services sleep, and the first request can take about a minute.
- **State is in memory.** Runs and approvals are held in memory by one process. A restart ends any run in progress, and the service cannot be scaled to several instances without moving this state to shared storage.
- **No user accounts.** The API has no authentication. Abuse is limited by per-client and global run limits, one writer per repository, and a time limit on paused runs, but these are not a substitute for real authentication. Setting a spending limit on the model key is recommended.
- **Rate limits of the model provider.** The free Groq tier limits tokens per minute. Retries with backoff are built in, but under heavy use a call can still fail with a rate-limit message. In that case the run reports it, and a missing explanation is replaced by a short fallback sentence.

Security

- Guardrails reduce risk but are rule-based, so a determined, cleverly worded request might get through. The two approval steps and the confinement of all file access to the repository are the main safety net.
- Test code from the model is executed on the server (in a temporary copy, with a time limit). That is acceptable for a demo with a fixed sample project, but a production system for arbitrary repositories would need a proper sandbox such as a container with no network access.